

## Sub-module 2

# AXI4-Lite Protocol

*Introduction & Architecture for PicoRV32 SoC Subsystem*

From master-slave channels to handshake protocol

# What We'll Cover

01 What is AXI4-Lite?

02 AXI4-Lite vs AXI4-Full

03 5 Channels Overview Table

04 Channel Signal Diagram

05 Write Transaction Flow (AW→W→B)

06 Read Transaction Flow (AR→R)

07 VALID/READY Rules

08 Lab: make axi\_sa

09 Summary

# What is AXI4-Lite?

AXI4-Lite (Advanced eXtensible Interface 4-Lite) is a simplified subset of ARM's AMBA AXI4 bus protocol. Designed for low-throughput, register-mapped control interfaces where simplicity and low-area are priorities over raw bandwidth.

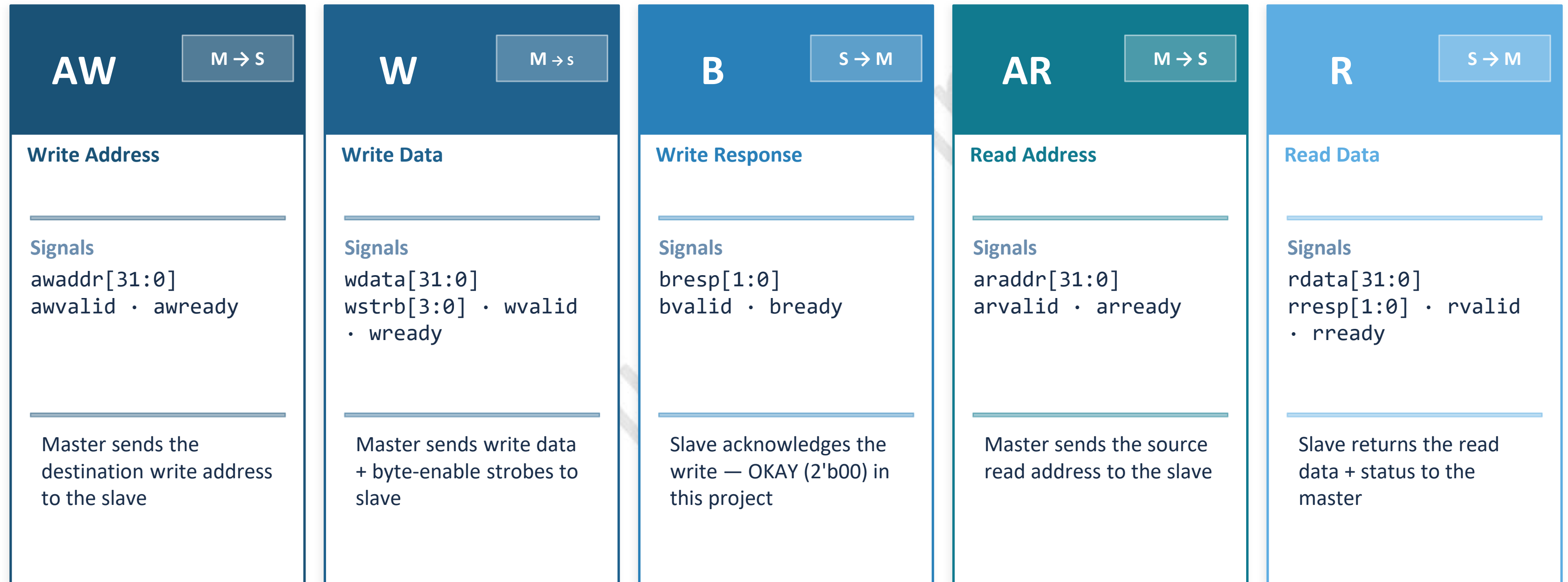
| Developed By                                                    | Transaction Type                                    | Address Space                                            | Use Case                                                               |
|-----------------------------------------------------------------|-----------------------------------------------------|----------------------------------------------------------|------------------------------------------------------------------------|
| ARM Ltd. (now Arm Holdings)<br>Part of the AMBA 4 specification | Single data-beat transfers only<br>No burst support | 32-bit or 64-bit address<br>32-bit data width (standard) | Control registers & status reads<br>Low-speed peripheral configuration |

# AXI4-Lite vs AXI4-Full

| Feature                 | AXI4-Lite           | AXI4-Full             |
|-------------------------|---------------------|-----------------------|
| Burst Transfers         | ✗ Not supported     | ✓ Up to 256 beats     |
| Transaction IDs         | ✗ None              | ✓ Multi-ID OoO        |
| Cache / Protect Signals | ✗ Not included      | ✓ Full signaling      |
| Register Access         | ✓ Ideal (single TX) | ✓ Supported           |
| Implementation Area     | ✓ Very small        | ✗ Large (queues, IDs) |
| Interconnect Lines      | ✓ ~35 signals       | ✗ 100+ signals        |

**Why AXI4-Lite for THIS SoC?** Single outstanding TX · Register-level peripheral access · No burst needed for UART/ROM · Minimal area (227-line interconnect)

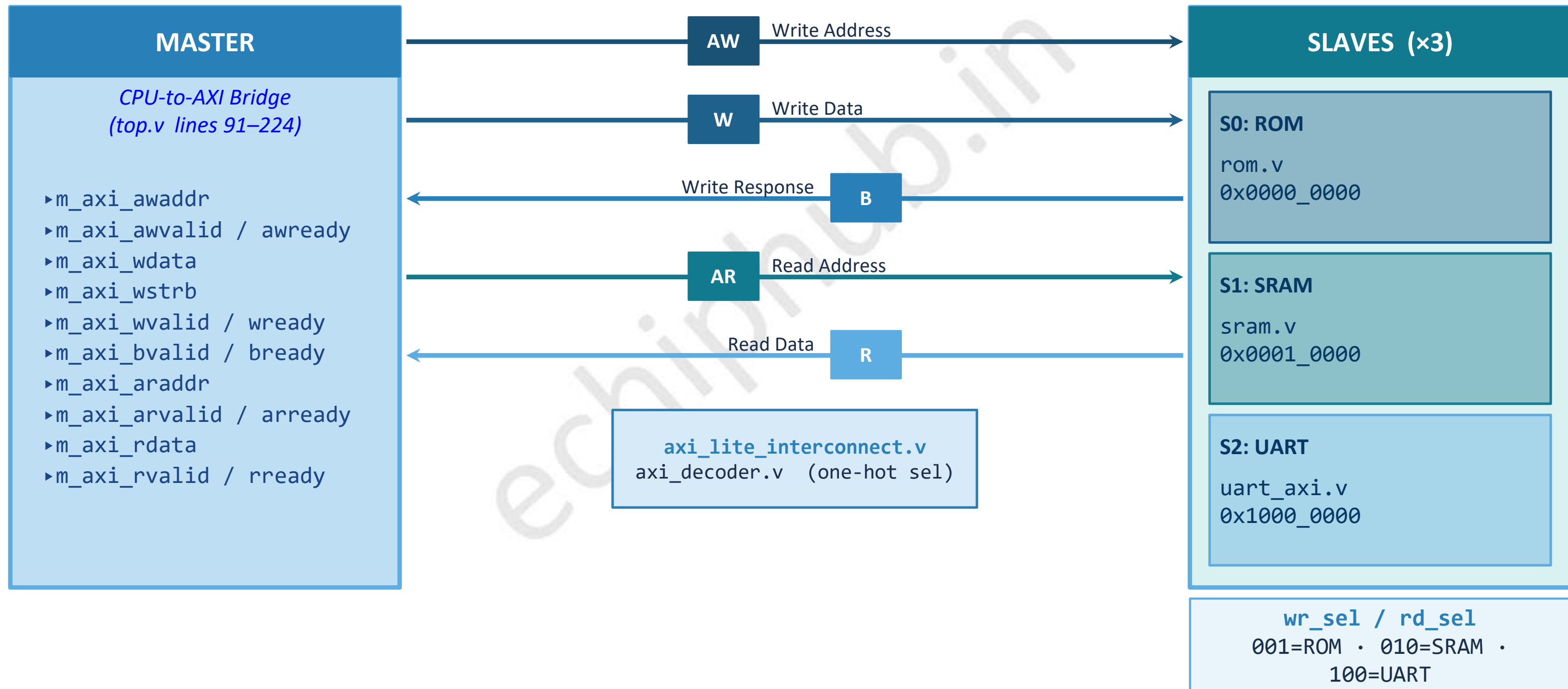
# The 5 AXI4-Lite Channels Overview



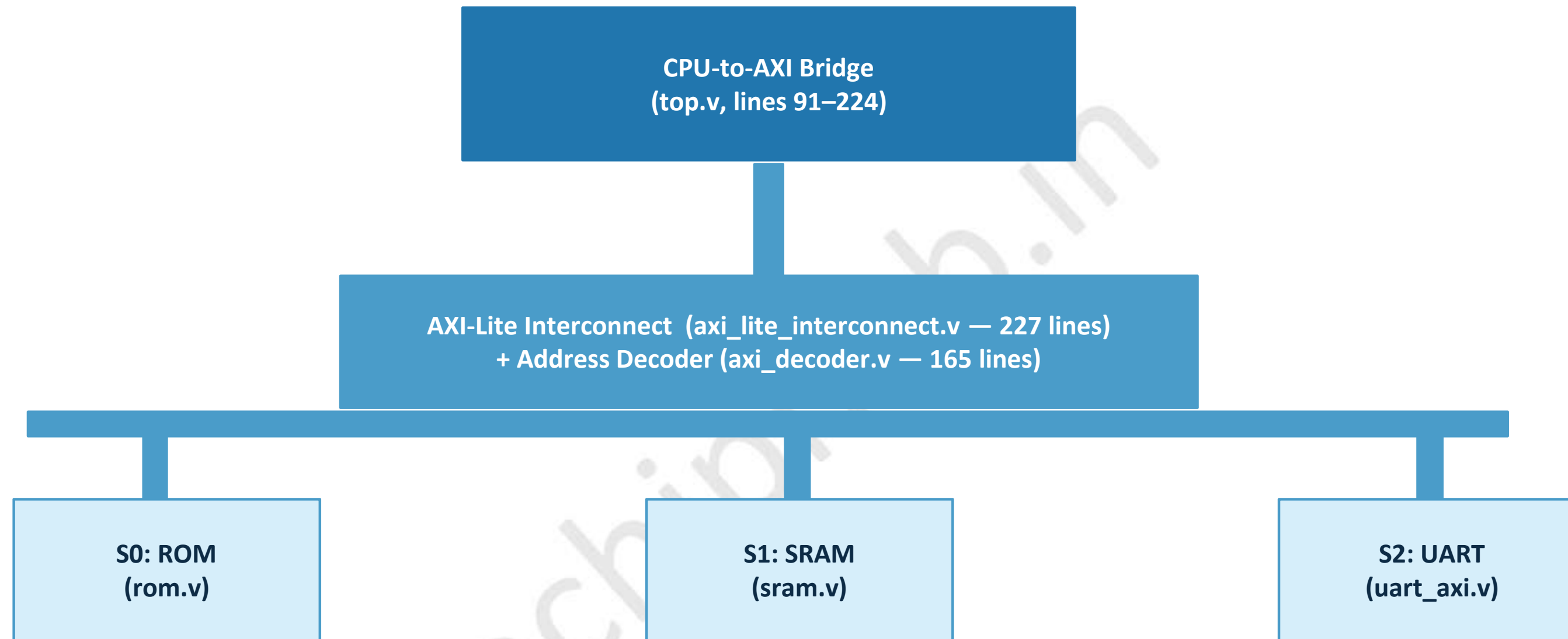
**Golden Rule:** A transfer occurs ONLY when VALID = 1 AND READY = 1 at the rising clock edge — on ALL five channels



# Channel Signal Diagram — THIS Project



# AXI4-Lite in THIS SoC — 1 Master, 3 Slaves



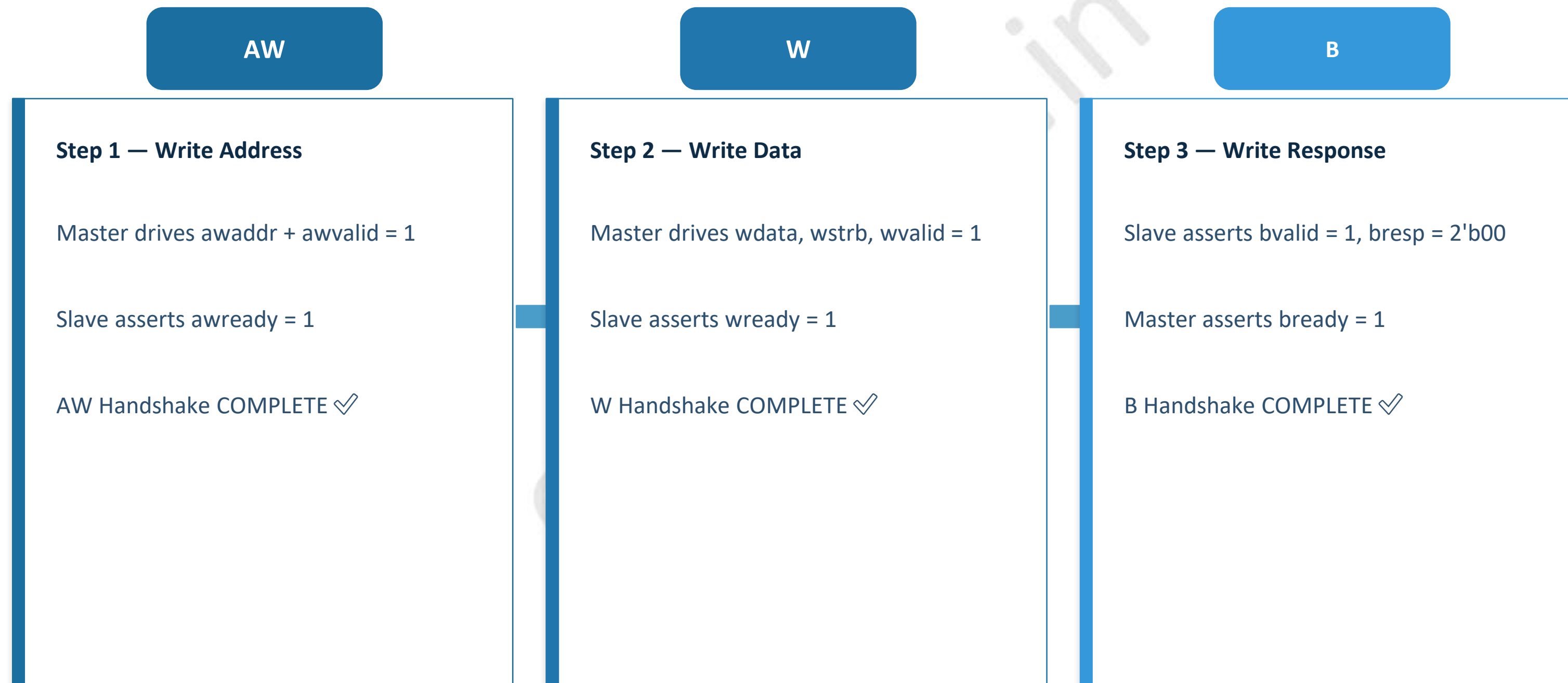
| Address Range | Slave                 | casez Pattern |
|---------------|-----------------------|---------------|
| 0x0000_????   | S0: ROM               | 32'h0000_???? |
| 0x0001_????   | S1: SRAM              | 32'h0001_???? |
| 0x1000_000?   | S2: UART              | 32'h1000_000? |
| default       | → ROM (safe fallback) | —             |

## Routing Signals

wr\_sel[2:0] — one-hot write select  
rd\_sel[2:0] — one-hot read select  
wr\_active / rd\_active

# Write Transaction : $AW \rightarrow W \rightarrow B$

A write transaction uses 3 channels in sequence: Write Address (AW)  $\rightarrow$  Write Data (W)  $\rightarrow$  Write Response (B)



✂ AW and W channels are independent — they can handshake simultaneously or in any order!



# AXI Write — CPU Sends 'H' to UART TX Register

Firmware: `uart_putc('H') → UART_TX = 0x1000_0000 → wdata = 0x0000_0048`

## Step 1: AW — Write Address Channel

CPU bridge → `awaddr = 0x1000_0000, awvalid = 1`

Interconnect → `wr_sel = 3'b100` (UART selected)

`uart_axi → awready = 1` (if not busy)

✓ AW Handshake complete

## Step 2: W — Write Data Channel

CPU bridge → `wdata = 0x0000_0048, wstrb = 4'h1, wvalid = 1`

`uart_axi → wready = 1` (if buffer not full)

`uart_axi captures: w_byte = 0x48, buf_valid = 1`

✓ W Handshake complete

## Step 3: B — Write Response Channel

`uart_axi → bvalid = 1, bresp = 2'b00` (OKAY)

CPU bridge → `bready = 1`

Bridge asserts `cpu_mem_ready = 1` → CPU resumes

✓ B Handshake complete — UART TX buffer = 0x48 ('H')

Result: UART TX buffer filled with 0x48 ('H') → `uart_tx` FSM serializes as 8N1 on `uart_tx` pin

# Read Transaction Flow: AR → R

A read transaction uses 2 channels: Read Address (AR) → Read Data (R). Simpler than write — no response channel needed.



## Timing (no wait-states):

Cycle 1: araddr=0x4, arvalid=1 → arready=1 (AR handshake)  
Cycle 2: rvalid=1, rdata=0xA5A5A5A5 → rready=1 (R handshake)

# AXI Read — CPU Polls UART Status Register

```
Firmware:  uart_getc()  →  poll  UART_STATUS (0x1000_0008)  &  0x2
```

## Step 1: AR — Read Address Channel

CPU bridge → araddr = 0x1000\_0008, arvalid = 1

Interconnect → rd\_sel = 3'b100 (UART selected)

uart\_axi → arready = 1

✓ AR Handshake complete

## Step 2: R — Read Data Channel

uart\_axi prepares: rdata = {30'b0, rx\_buf\_valid, 1'b0}

uart\_axi → rvalid = 1, rdata = 0x0000\_0002 (RX ready)

CPU bridge → rready = 1

✓ R Handshake complete

Firmware checks: (UART\_STATUS & 0x2) != 0 → data available in RX buffer

# VALID/READY Handshake Rules

**GOLDEN RULE:** Transfer occurs **ONLY** when **VALID = 1 AND READY = 1** at the rising clock edge

1

VALID may be asserted without READY. The sender waits.

2

READY may be asserted without VALID. The receiver is ready to accept.

3

Once VALID is asserted, it **MUST NOT** be de-asserted until the handshake completes.

4

READY can be de-asserted freely between transactions.

5

Master must **NOT** make VALID dependent on READY (no circular deadlock).

6

Transfer data is captured by the receiver on the handshake clock edge.

# WSTRB — Write Strobe Byte Enable

## What is WSTRB?

4-bit signal (one bit per byte lane) indicating which bytes of wdata[31:0] are valid. Allows sub-word writes without a read-modify-write cycle.

| WSTRB bit | Byte lane | wdata bits |
|-----------|-----------|------------|
| wstrb[0]  | Byte 0    | [7:0]      |
| wstrb[1]  | Byte 1    | [15:8]     |
| wstrb[2]  | Byte 2    | [23:16]    |
| wstrb[3]  | Byte 3    | [31:24]    |

## Project Examples — Firmware → WSTRB Mapping

**SW**    `x0, 0 (UART_BASE)`

**WSTRB** = `4'hF`

Full 32-bit word write — all 4 bytes valid

**SB**    `x0, 0 (UART_BASE)`

**WSTRB** = `4'h1`

Byte write — only wdata[7:0] (byte 0) is valid

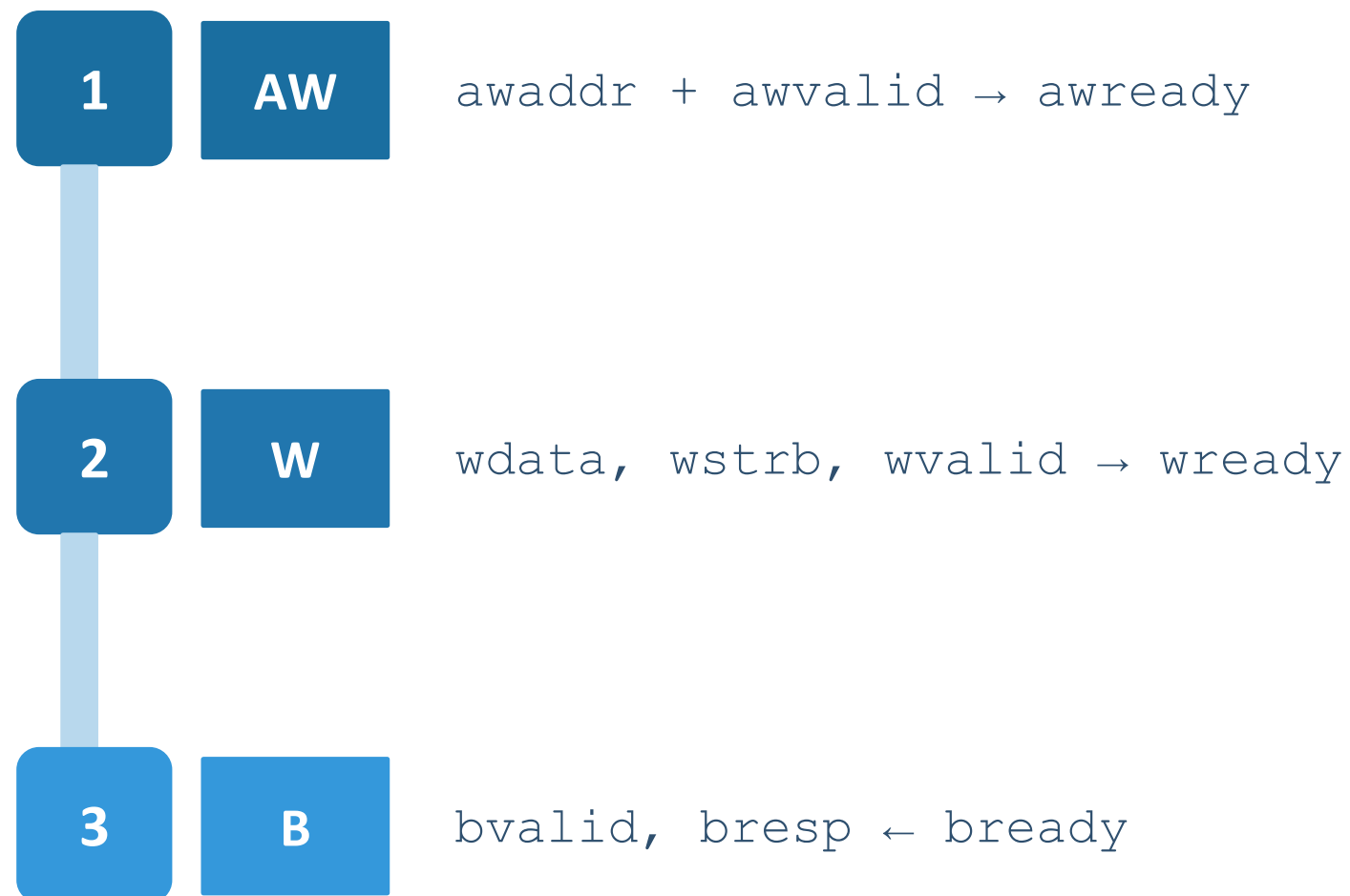
## uart\_axi.v — byte selection from WSTRB:

```
if      (s_axi_wstrb[0]) w_byte <= s_axi_wdata[ 7: 0];  
else if (s_axi_wstrb[1]) w_byte <= s_axi_wdata[15: 8];
```

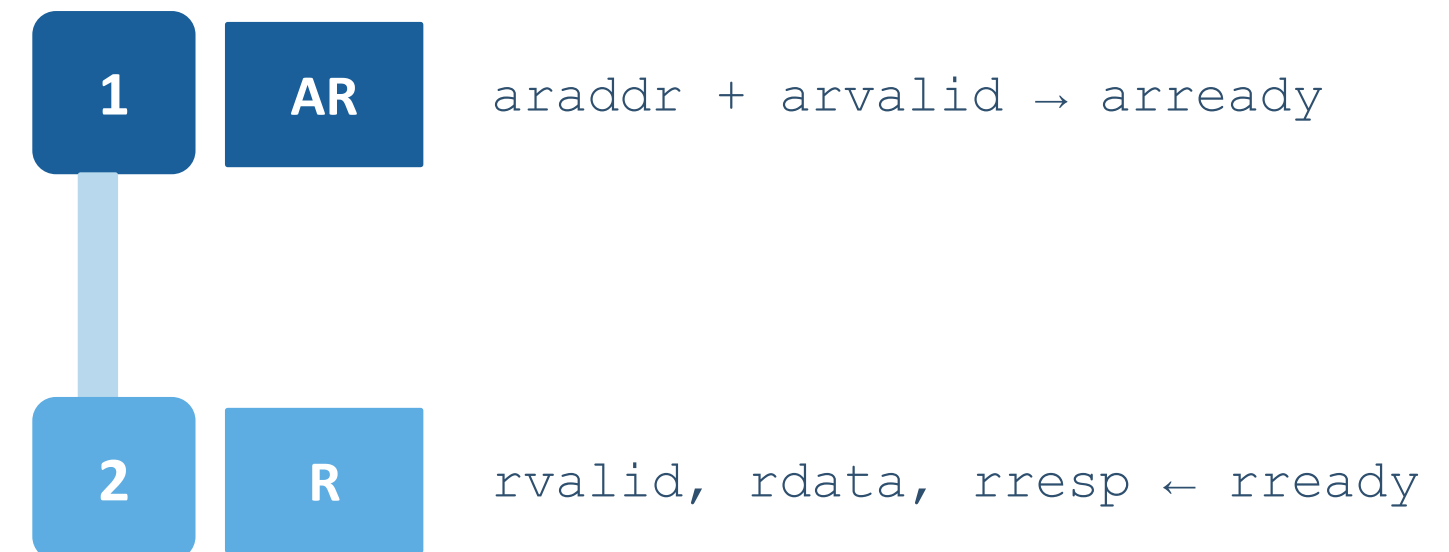


# Write & Read Transaction Flow Summary

## WRITE Transaction



## READ Transaction



**RESP Encoding:** 2'b00 = OKAY (all transactions in this project) 2'b10 = SLVERR (slave error) 2'b11 = DECERR (decode error — check address map)

# Why AXI4-Lite for THIS SoC?



## Simplicity

No burst logic — perfect for UART/ROM serving one word at a time. Simple FSM in top.v.



## Decoupling

AW and W channels let the CPU pipeline address and data separately, improving throughput.



## Reuse

All 3 slaves (ROM, SRAM, UART) share one 35-signal AXI interface via the interconnect.



## Area

rtl/axi\_lite\_interconnect.v = 227 lines — extremely lightweight for a 1M→3S fabric.



## Verification

Independent channels enable isolated testing: make axi\_sa tests handshake alone without RTL.



## Industry Std.

ARM AMBA AXI4-Lite is the go-to choice for register-mapped peripherals in RISC-V SoC designs.

# Lab: make axi\_sa — Standalone AXI-Lite Handshake Demo

```
$ make axi_sa → ./obj_axi_sa/sim_axi_sa → tb_axi_lite_standalone.vcd (3.8 KB)
```

## Expected Terminal Output:

```
[WRITE] Address = 0x00000004  
[WRITE] Data      = 0xa5a5a5a5  
[WRITE] awready=1 → Handshake Done  
[WRITE] bvalid=1  → Write Response Received (OKAY)  
[READ]  Address = 0x00000004  
[READ]  rvalid=1 → Data Received = 0xa5a5a5a5  
[CHECK] Write Data == Read Data → PASS ✓
```

## GTKWave Signals:

```
clk, reset  
awaddr[31:0], awvalid, awready  
wdata[31:0], wstrb[3:0], wvalid, wready  
bvalid, bready, bresp[1:0]  
araddr[31:0], arvalid, arready  
rdata[31:0], rvalid, rready
```

## All Sub-module 2 Lab Commands:

| Command        | Output VCD/FST             | Tests              |
|----------------|----------------------------|--------------------|
| make axi_sa    | tb_axi_lite_standalone.vcd | AXI handshake only |
| make axi_uart  | tb_axi_lite.vcd            | AXI + UART slave   |
| make riscv_axi | tb_riscv_axi_lite.vcd      | CPU bridge + AXI   |
| make soc       | tb_top.fst                 | Complete SoC       |



# Key Takeaways

5 Channels: AW (write addr) + W (write data) + B (write resp) + AR (read addr) + R (read data)

Golden Rule: Transfer occurs ONLY when VALID AND READY = 1 at rising clock edge

THIS Project: 1 CPU bridge master  $\leftrightarrow$  3 slaves via axi\_decoder.v one-hot select

Address Map: ROM = 0x0000\_0000 | SRAM = 0x0001\_0000 | UART = 0x1000\_0000

Lab: make axi\_sa  $\rightarrow$  write 0xA5A5A5A5  $\rightarrow$  read back  $\rightarrow$  PASS  $\checkmark$

WSTRB: firmware SB  $\rightarrow$  WSTRB=4'h1  $\rightarrow$  uart\_axi selects wdata[7:0] as TX byte